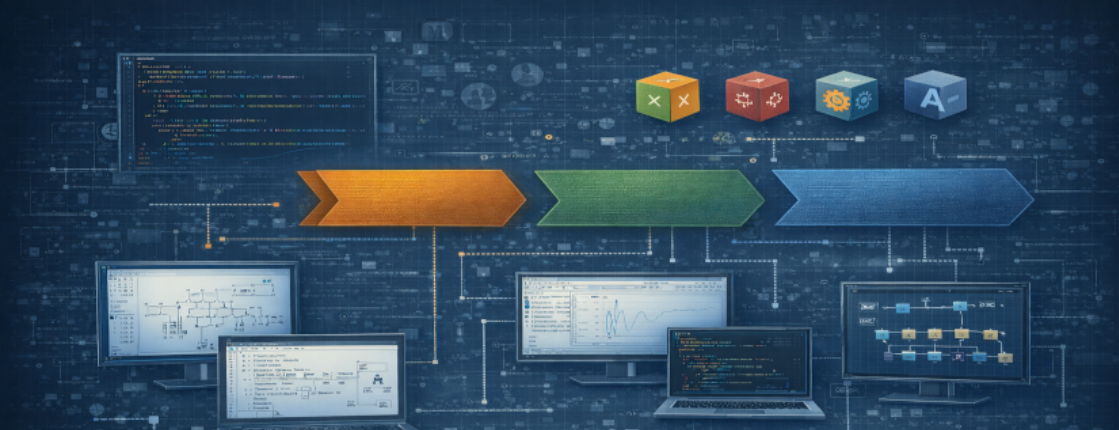




5. Linguagens, Ambientes e Simuladores de Modelagem e Simulação

5.1 Introdução

No capítulo anterior, foi discutida a construção do modelo conceitual e a escolha do paradigma de modelagem. Com isso, o leitor já dispõe dos elementos necessários para decidir *o que* modelar, *por que* modelar e *segundo que semântica* representar o sistema. Resta, entretanto, uma questão decisiva para a continuação do projeto: por meio de que tecnologia computacional esse modelo será descrito, implementado, executado, depurado e analisado?

Essa questão não admite resposta única, porque a prática contemporânea da modelagem e simulação recorre a artefatos computacionais de naturezas bastante distintas. Em alguns casos, o modelo é implementado em uma linguagem de programação de propósito geral, apoiado por bibliotecas especializadas. Em outros, utiliza-se uma linguagem clássica de simulação. Em outros ainda, o modelador trabalha com linguagens declarativas, diagramas gráficos, ambientes multidomínio ou plataformas híbridas de construção e execução de modelos. Por isso, este capítulo tem uma função essencialmente instrumental: ele não substitui a construção conceitual já realizada, mas apresenta os meios computacionais pelos quais essa construção se torna executável.

Essa diversidade tecnológica exige também precisão terminológica. Na prática, é co-

um tratar como equivalentes expressões como *linguagem*, *biblioteca*, *framework*, *simulador*, *plataforma* e *ambiente*. Contudo, essas categorias não são idênticas. Uma linguagem de programação de propósito geral não é, por si só, uma linguagem de simulação. Uma biblioteca de simulação não coincide com um ambiente de modelagem. Um ambiente gráfico tampouco se reduz a uma linguagem, embora frequentemente incorpore uma linguagem subjacente, explícita ou implícita.

Também é importante registrar, já no início do capítulo, a política editorial adotada neste livro. Os ambientes e simuladores apresentados aqui incluem Arena, Ptolemy II, OpenModelica, Scilab/Xcos e GenESyS. Entretanto, os exemplos práticos recorrentes dos capítulos posteriores deste volume utilizarão apenas Ptolemy II, OpenModelica e Arena. Scilab/Xcos e GenESyS continuam relevantes para o panorama metodológico e histórico da área e, por essa razão, permanecem neste capítulo. No caso particular do GenESyS, sua apresentação geral e seu primeiro contato de uso serão mantidos aqui, mas seu tratamento aprofundado ficará reservado a um segundo livro específico.

A figura 5.1 explicita a posição deste capítulo no fluxo geral da obra. O modelo conceitual e o paradigma já foram escolhidos; o capítulo atual apresenta os meios computacionais pelos quais essa escolha poderá ser materializada. O capítulo seguinte, por sua vez, voltará a atenção para os dados necessários à parametrização do modelo.

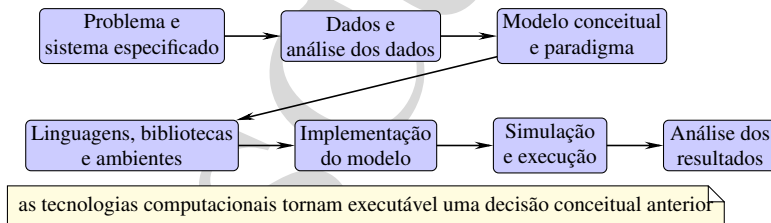


Figura 5.1: Papel das linguagens, ambientes e simuladores no fluxo metodológico do estudo de simulação.

Exemplo 5.1 Explique por que a escolha de um simulador não deve anteceder a escolha do paradigma de modelagem.

Solução: O simulador é um meio de implementação, enquanto o paradigma de modelagem define a semântica da representação do sistema, isto é, como tempo, estado, dinâmica e interação serão compreendidos. Se o simulador for escolhido antes do paradigma, corre-se o risco de forçar o problema a uma tecnologia familiar, em vez de selecionar a forma de modelagem mais adequada aos objetivos do estudo. ■

5.2 Taxonomia de tecnologias computacionais para modelagem e simulação

Para evitar ambiguidades, adotaremos neste capítulo uma classificação em cinco categorias principais: linguagens de programação de propósito geral, bibliotecas e *frameworks* de simulação, linguagens clássicas de simulação, linguagens declarativas de modelagem e ambientes de modelagem e simulação. Essa classificação não é apenas didática. Ela reflete diferenças reais de abstração, expressividade, papel do usuário e responsabilidades assumidas pela tecnologia empregada.

A figura 5.2 sintetiza essa organização. Nela, pode-se observar que as categorias se relacionam, mas não se confundem. Certas ferramentas ocupam posição intermediária ou combinada nesse espectro, mas isso não reduz a utilidade metodológica da distinção.

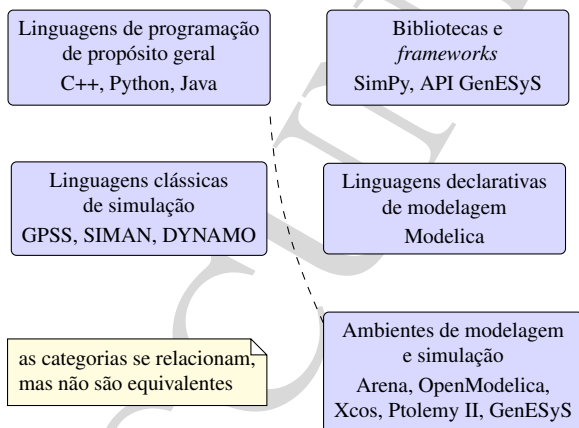


Figura 5.2: Taxonomia de tecnologias computacionais para modelagem e simulação.

5.2.1 Linguagens de programação de propósito geral

Linguagens de programação de propósito geral, como C++, Java ou Python, não foram criadas especificamente para simulação. Ainda assim, podem ser usadas para implementar modelos, desde que o desenvolvedor construa ou incorpore mecanismos apropriados para controle do tempo, representação do estado, agendamento de eventos, geração de números pseudoaleatórios, coleta de estatísticas e organização do plano experimental.

A principal vantagem dessa abordagem está na flexibilidade. Em princípio, qualquer estrutura computável pode ser construída. Isso é especialmente valioso quando o modelo exige algoritmos específicos, forte integração com outros sistemas, alto desempenho ou controle detalhado da implementação. Sua principal desvantagem, por outro lado, é o custo de desenvolvimento. Sem apoio adicional, grande parte da infraestrutura de simulação precisa ser

explicitamente construída no projeto.

5.2.2 Bibliotecas e *frameworks* de simulação

Bibliotecas e *frameworks* de simulação ocupam posição intermediária entre linguagens de propósito geral e linguagens de simulação propriamente ditas. Em vez de criar uma nova linguagem independente, elas ampliam uma linguagem hospedeira com abstrações e mecanismos voltados à modelagem e à execução de simulações. Em bibliotecas desse tipo, o modelador continua programando na linguagem base, mas passa a dispor de componentes para representar entidades, processos, eventos, recursos, filas, sinais, equações, animação ou outros elementos recorrentes.

Essa estratégia é bastante comum em abordagens contemporâneas. Ela reduz o esforço de implementação, preserva a flexibilidade da linguagem hospedeira e facilita integração com bibliotecas numéricas, bases de dados, sistemas legados e ferramentas estatísticas. Ao mesmo tempo, exige do usuário maior familiaridade com a linguagem subjacente e com as convenções arquiteturais do *framework* adotado.

5.2.3 Linguagens clássicas de simulação

Linguagens clássicas de simulação são linguagens criadas especificamente para descrever modelos de simulação e, em muitos casos, também o plano experimental associado. Historicamente, esse grupo inclui GPSS, SIMSCRIPT, GASP, DYNAMO e SIMAN. Essas linguagens tiveram papel decisivo na consolidação da área e moldaram muitos conceitos ainda presentes em simuladores contemporâneos.

Sua principal característica é oferecer estruturas sintáticas diretamente orientadas a elementos de modelagem, como entidades, transações, filas, recursos, atrasos, eventos, processos ou equações de evolução. Em comparação com o uso de linguagens de propósito geral, elas tornam explícita a lógica da simulação em um nível mais próximo do domínio do problema. Mesmo quando hoje não são mais a ferramenta principal do modelador, continuam sendo importantes como base histórica e conceitual.

5.2.4 Linguagens declarativas de modelagem

Linguagens declarativas de modelagem se distinguem por descrever relações, componentes, conectores, equações e restrições, em vez de programar imperativamente a ordem das operações numéricas. Nesse estilo, o foco recai sobre *o que* o sistema é e *como* suas partes se relacionam, deixando para o ambiente de modelagem a tarefa de reorganizar essa descrição em estrutura executável.

Essa abordagem é especialmente adequada para sistemas físicos e multidomínio, nos quais a representação por componentes e equações oferece forte clareza conceitual. A linguagem Modelica é o exemplo mais importante desta categoria no contexto do livro.

5.2.5 Ambientes de modelagem e simulação

Ambientes de modelagem e simulação reúnem, em uma mesma infraestrutura, editor textual e/ou gráfico, bibliotecas de componentes, motor de execução, mecanismos de depuração, parametrização experimental, visualização de resultados e, em muitos casos, apoio à análise estatística ou à otimização. Eles reduzem o custo de construção do modelo, facilitam experimentação e aproximam o processo de modelagem das necessidades de engenheiros, cientistas e analistas de domínio.

É justamente nessa categoria que se situam grande parte das ferramentas mais usadas em ensino, pesquisa e prática profissional. Contudo, é importante não confundir ambiente com formalismo. Um mesmo ambiente pode suportar diferentes paradigmas, e um mesmo formalismo pode aparecer em ambientes distintos.

5.3 Linguagens de simulação e de modelagem representativas

A literatura e a prática da área produziram dezenas de linguagens e formas de descrever modelos. Em vez de buscar abrangência enciclopédica, este capítulo seleciona algumas linguagens particularmente representativas para os objetivos da obra: GPSS e SIMAN, por sua importância histórica e metodológica na simulação discreta; Modelica, por sua centralidade na modelagem declarativa multidomínio; e GenESyS, por seu interesse pedagógico e por sua posição particular no ecossistema discutido neste capítulo.

5.3.1 GPSS

GPSS (*General Purpose Simulation System*) é uma das linguagens mais emblemáticas da tradição da simulação discreta orientada a transações. Sua importância histórica está em cristalizar uma forma de pensar sistemas discretos em termos de fluxo de entidades por blocos que representam operações como geração, fila, apreensão de recursos, atraso, liberação e término.

A listagem 5.1 mostra um exemplo simples de barbearia descrito em GPSS. Cada comando corresponde a uma etapa pela qual as entidades passam, desde a chegada ao sistema até sua saída. O valor didático desse tipo de formulação está em tornar muito explícita a lógica básica da modelagem por fluxo.

```
1 SIMULATE
2 * Model segment 1
3 GENERATE 18,6
4 QUEUE Chairs
5 SEIZE Joe
6 DEPART Chairs
7 ADVANCE 16,4
8 RELEASE Joe
9 TERMINATE
```

```
10      * Model segment 2
11      GENERATE 480
12      TERMINATE 1
13      * Control cards
14      START      1
15      END
```

Listing 5.1: Modelo simples de barbearia em GPSS

Mesmo quando não é mais a ferramenta principal do modelador contemporâneo, GPSS continua sendo importante como referência histórica e pedagógica, pois explicita a estrutura subjacente de muitos modelos discretos modernos.

5.3.2 SIMAN

SIMAN (*SIMulation ANalysis*) ocupa posição especialmente relevante nesta obra porque estabelece uma ponte histórica e conceitual entre a tradição textual das linguagens clássicas e a tradição dos ambientes gráficos modernos de simulação discreta. É também particularmente importante porque sua semântica está na base de ambientes como o Arena.

Uma de suas contribuições metodológicas mais elegantes é a separação entre *estrutura do modelo* e *estrutura experimental*. Essa distinção reforça algo fundamental em simulação: o modelo e o experimento sobre o modelo não são o mesmo artefato. O primeiro descreve a lógica do sistema; o segundo descreve as condições sob as quais o modelo será executado.

A figura 5.3 sintetiza essa relação entre linguagem clássica e ambiente gráfico posterior.

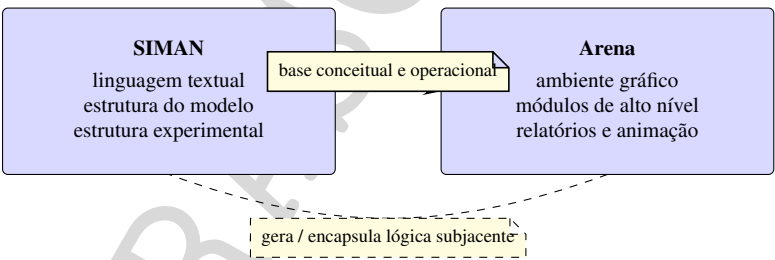


Figura 5.3: SIMAN como ponte conceitual entre linguagens clássicas de simulação e o ambiente gráfico Arena.

A listagem 5.2 mostra um trecho de estrutura de modelo em SIMAN, enquanto a listagem 5.3 mostra parte da estrutura experimental correspondente.

```
1      $      CREATE,      1,SecondstoBaseTime(0.0),Entity
           1:SecondstoBaseTime(expo(20)):NEXT(6$);
2      6$      ASSIGN:      Create 1.NumberOut=Create 1.NumberOut + 1;
3      0$      QUEUE,      Seize 1.Queue;
```

```

4 SEIZE,      2,Other:    MACHINE,1:NEXT(10$);
5 10$        DELAY:      0.0,,VA:NEXT(2$);
6 2$         DELAY:      UNIF(1,2),,Other:NEXT(3$);
7 3$         RELEASE:    MACHINE,1:NEXT(4$);
8 4$         ASSIGN:     Dispose 1.NumberOut=Dispose 1.NumberOut + 1;
9 11$        DISPOSE:    Yes;

```

Listing 5.2: Exemplo de estrutura de modelo em SIMAN

```

1 PROJECT, "Unnamed Project", "-",,,Yes,Yes,Yes,Yes,Yes,Yes,Yes,Yes,No,No;
2 VARIABLES: Dispose 1.NumberOut, CLEAR(Statistics), CATEGORY("Exclude"):
3 Create 1.NumberOut,CLEAR(Statistics),CATEGORY("Exclude");
4 QUEUES: Seize 1.Queue,FIFO,,AUTOSTATS(Yes,,);
5 RESOURCES: MACHINE, Capacity(1),,, COST(0.0,0.0,0.0),
6 CATEGORY(Resources),,, AUTOSTATS(Yes,,);
7 REPLICATE, 30 ,,, SecondsToBaseTime(3600), Yes,Yes,,, 24, Seconds, No,
8 No,,, Yes;
9 ENTITIES: Entity 1, Picture.Report, 0.0,0.0,0.0,0.0,0.0,0.0,
10 AUTOSTATS(Yes,,);

```

Listing 5.3: Exemplo de plano experimental em SIMAN

Exemplo 5.2 Explique a importância metodológica da separação entre estrutura do modelo e estrutura experimental em SIMAN.

Solução: Essa separação permite distinguir com clareza entre a lógica do sistema modelado e as condições sob as quais o modelo será executado. Com isso, diferentes experimentos podem ser conduzidos sobre a mesma estrutura do modelo sem que a representação do sistema precise ser reescrita. Metodologicamente, isso reforça a distinção entre representar o sistema e projetar a análise experimental sobre ele. ■

5.3.3 Modelica

Modelica ocupa posição singular na história das linguagens de modelagem e simulação. Diferentemente das linguagens clássicas orientadas principalmente a eventos discretos, fluxos de transações ou experimentos operacionais, Modelica foi concebida como linguagem declarativa, orientada a objetos, voltada à modelagem de sistemas físicos grandes, complexos e heterogêneos. Seu foco principal está na descrição de componentes, conectores, equações e relações entre domínios físicos distintos, como mecânica, elétrica, térmica, hidráulica, controle e energia.

Do ponto de vista do usuário, um modelo em Modelica é frequentemente percebido inicialmente como diagrama de componentes conectados. Internamente, porém, cada componente é descrito em linguagem textual por equações, declarações e estruturas orientadas a objetos. Essa característica é central para compreender sua relevância metodológica. Em vez de exigir que o modelador estabeleça previamente uma direção causal para cada cálculo, Modelica

permite que o sistema seja descrito por relações declarativas. Em seguida, o ambiente de simulação reorganiza essa descrição em estrutura executável.

A figura 5.4 ilustra esse estilo de modelagem baseada em componentes e conexões.

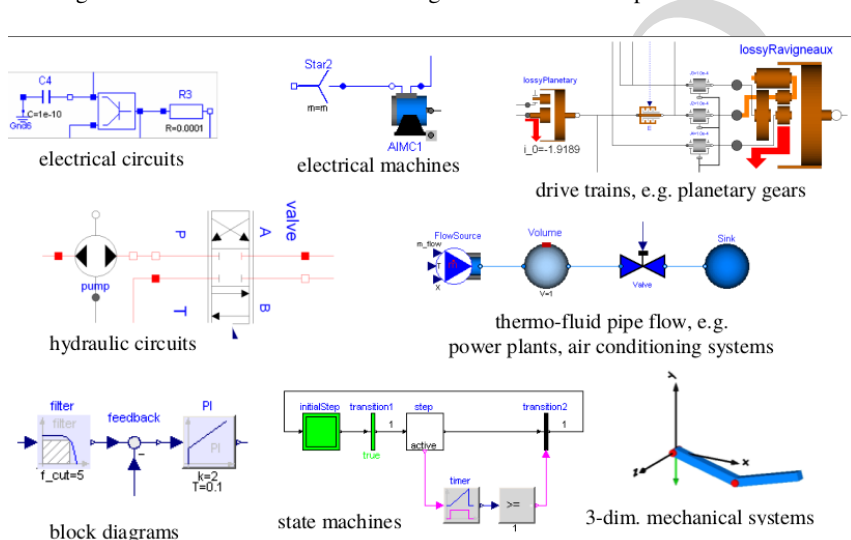


Figura 5.4: Exemplos de diagramas de objetos em Modelica para diferentes domínios físicos e funcionais.

A listagem 5.4 apresenta um circuito simples em Modelica. Nela, o foco está em declarar componentes e conexões, e não em programar imperativamente a ordem de execução numérica.

```

1  model circuit
2  Resistor R1(R=10);
3  Capacitor C(C=0.01);
4  Resistor R2(R=100);
5  Inductor L(L=0.1);
6  VsourceAC AC;
7  Ground G;
8  equation
9  connect(AC.p, R1.p);
10 connect(R1.n, C.p);
11 connect(C.n, AC.n);
12 connect(R1.p, R2.p);
13 connect(R2.n, L.p);
14 connect(L.n, C.n);
15 connect(AC.n, G.p);
16 end circuit;

```


Listing 5.4: Modelo de circuito eletrônico em Modelica

A grande vantagem dessa formulação aparece quando comparada à construção causal explícita de um diagrama de blocos tradicional. Em Modelica, a mesma lei física pode ser usada em contextos distintos sem que o usuário precise decidir antecipadamente em que direção resolver cada equação. Essa propriedade torna a linguagem particularmente poderosa para sistemas multidomínio e bibliotecas reutilizáveis de componentes físicos.

Exemplo 5.3 Explique por que Modelica é caracterizada como linguagem declarativa e acausal, e não como linguagem imperativa de programação numérica.

Solução: Modelica é declarativa porque o modelador especifica componentes, conexões e equações que descrevem o sistema, em vez de programar explicitamente a sequência de operações numéricas. Ela é acausal porque as relações entre variáveis são descritas por equações, sem que o usuário precise fixar antecipadamente a direção causal de cada cálculo. O ambiente de simulação é responsável por analisar essa estrutura e derivar uma forma executável do modelo. ■

5.3.4 GenESyS

GenESyS (*GENeric and EXpansible System Simulator*) ocupa, neste capítulo, uma posição particular. Ele não é apenas um simulador ou um ambiente de modelagem; também fornece uma linguagem textual própria para descrição de modelos e, adicionalmente, pode ser usado como biblioteca de simulação em C++. Essa dupla natureza o torna particularmente interessante do ponto de vista pedagógico, pois permite observar três níveis de utilização de uma mesma infraestrutura: modelagem por declaração textual, modelagem por programação direta sobre a API da biblioteca e modelagem por interface gráfica.

Historicamente, o GenESyS foi concebido com forte inspiração em SIMAN e Arena, especialmente na tradição da modelagem discreta orientada a eventos e na ideia de blocos de fluxo como Create, Seize, Delay, Release e Dispose. Ao mesmo tempo, foi projetado para ser expansível, permitindo incorporar novas técnicas de modelagem e novos componentes. Essa característica justifica sua presença neste capítulo, embora ele não seja usado como plataforma prática recorrente no restante deste volume.

A listagem 5.5 mostra um exemplo de declaração textual simples em GenESyS.

```
1 0 SimulatorInfo version="19.10 (Halloween19)"
2 0 ModelInfo name="Model 1" replicationLength=3600.000000
   numberOfReplications=30
3 1 EntityType name="Any Entity"
4 2 Resource name="Machine" capacity=1
5 3 Queue name="Queue_Machine" orderRule=1
6 4 Create entityTypeName="Any Entity" name="Create 1" nextId0=5
   timeBetweenCreations="expo(20)"
```

```

7      5 Seize name="Seize 1" queueName="Queue_Machine" resourceType=1
          resourceName="Machine" nextId0=6
8      6 Delay name="Delay 1" delayExpression="unif(1,2)" nextId0=7
9      7 Release name="Release 1" queueName="Queue_Machine"
          resourceName="Machine" nextId0=8 rule=4
10     8 Dispose name="Dispose 1"

```

Listing 5.5: Exemplo de declaração textual de modelo em GenESyS

Como biblioteca em C++, o mesmo modelo pode ser construído programaticamente, como mostra a listagem 5.6.

```

1      #include "Simulator.h"
2      #include "Create.h"
3      #include "Resource.h"
4      #include "Queue.h"
5      #include "Seize.h"
6      #include "Delay.h"
7      #include "Release.h"
8      #include "Dispose.h"
9
10     int main(int argc, char** argv) {
11         Simulator* simulator = new Simulator();
12         Model* model = new Model(simulator);
13
14         ModelInfo* infos = model->infos();
15         infos->setReplicationLength(3600);
16         infos->setNumberOfReplications(30);
17
18         EntityType* entityType1 = new EntityType(model, "Any_Entity");
19         Create* create1 = new Create(model);
20         create1->setEntityType(entityType1);
21         create1->setTimeBetweenCreationsExpression("expo(20)");
22
23         Resource* machine1 = new Resource(model, "Machine");
24         Queue* queueSeize1 = new Queue(model, "Queue_Machine");
25
26         Seize* seize1 = new Seize(model);
27         seize1->setResource(machine1);
28         seize1->setQueue(queueSeize1);
29
30         Delay* delay1 = new Delay(model);
31         delay1->setDelayExpression("unif(1,2)");
32
33         Release* release1 = new Release(model);
34         release1->setResource(machine1);
35
36         Dispose* dispose1 = new Dispose(model);
37
38         create1->nextComponents()->insert(seize1);

```

```
39     seize1->nextComponents()->insert(delay1);
40     delay1->nextComponents()->insert(release1);
41     release1->nextComponents()->insert(dispose1);
42
43     simulator->models()->insert(model);
44     model->simulation()->start();
45     model->simulation()->showReports();
46     simulator->~Simulator();
47 }
```

Listing 5.6: Exemplo de construção programática de modelo com a biblioteca GenESyS

A principal observação editorial aqui é a seguinte: GenESyS continua sendo importante para o panorama da área, para a comparação entre linguagens e ambientes e para o entendimento arquitetural de uma plataforma expansível de simulação. Entretanto, sua exploração aprofundada, bem como seu uso prático recorrente, ficam reservados ao segundo livro dedicado especificamente a essa plataforma.

5.4 Ambientes e simuladores relevantes para a obra

Se as linguagens e bibliotecas definem ou apoiam a forma de descrever modelos, os ambientes e simuladores fornecem a infraestrutura computacional que torna essa descrição executável. Em termos gerais, um simulador é um programa capaz de interpretar, traduzir ou executar um modelo e produzir resultados correspondentes a uma ou mais corridas de simulação. Um ambiente de modelagem e simulação, por sua vez, combina esse motor de execução com ferramentas adicionais, como editor gráfico, bibliotecas de componentes, depuração, parametrização experimental, visualização e geração de relatórios.

5.4.1 Scilab e Xcos

Scilab é um ambiente aberto de computação numérica e programação científica. No contexto desta obra, o interesse principal no Scilab não está apenas em sua linguagem, mas também no fato de incluir o Xcos, um ambiente gráfico para modelagem e simulação de sistemas dinâmicos contínuos, discretos e híbridos.

Do ponto de vista conceitual, o Xcos é um editor gráfico de diagramas de blocos. Seu papel é aproximar o modelador de uma representação visual da estrutura do sistema, permitindo que blocos, integradores, ganhos, somadores, fontes e elementos de controle sejam conectados diretamente na área de edição. Isso o torna particularmente útil em contexto didático, pois facilita a transição entre modelo matemático e estrutura executável.

É importante, contudo, distinguir o papel do Scilab e do Xcos. O Scilab é o ambiente computacional mais amplo, com linguagem própria, bibliotecas numéricas e ferramentas de análise. O Xcos é um subsistema gráfico voltado à modelagem por diagramas de blocos e

à simulação de sistemas híbridos. Em outras palavras, o Xcos não substitui o Scilab; ele o complementa.

Uma vista típica do ambiente pode ser observada na Figura 5.5.

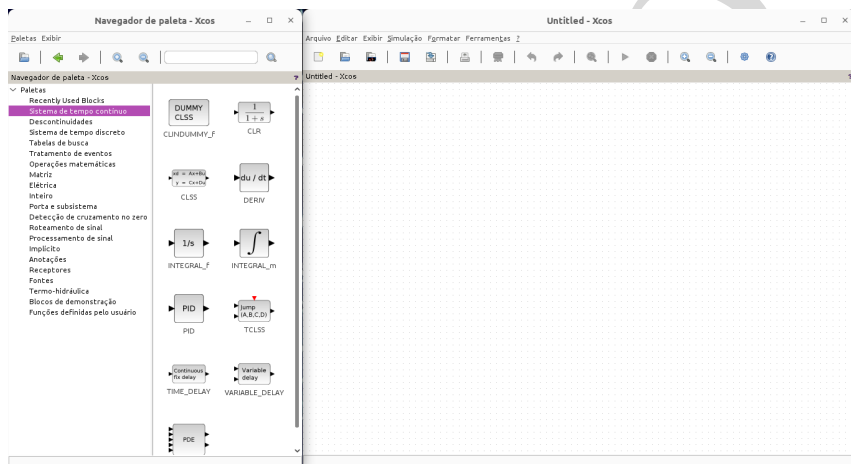


Figura 5.5: Tela inicial do ambiente Xcos.

Como primeiro exemplo, considere o modelo contínuo de disseminação de doença apresentado na Figura 5.6. O sistema descreve uma variante simples do modelo SIR, com três variáveis de estado x_1 , x_2 e x_3 , representando frações de suscetíveis, infectados e recuperados, respectivamente.

Ao executar a simulação, obtém-se o comportamento mostrado na Figura 5.7. O exemplo ilustra de forma clara a relação entre equações diferenciais, estrutura gráfica e dinâmica simulada.

Do ponto de vista editorial deste livro, Scilab/Xcos é importante como ambiente representativo e como ponte didática entre modelagem matemática e simulação gráfica. Contudo, ele não será o ambiente recorrente dos capítulos práticos posteriores.

5.4.2 Ptolemy II

Ptolemy II é um ambiente de modelagem e simulação de código aberto particularmente importante para esta obra porque oferece visão explícita de *modelos de computação* e de composição heterogênea entre domínios. Essa característica o torna especialmente relevante para os capítulos que tratarão de modelagem contínua, máquinas de estados finitos e sistemas híbridos.

Em Ptolemy II, os componentes do modelo são chamados de atores. Atores se comunicam por meio de portas interconectadas e podem ser organizados em modelos hierárquicos.

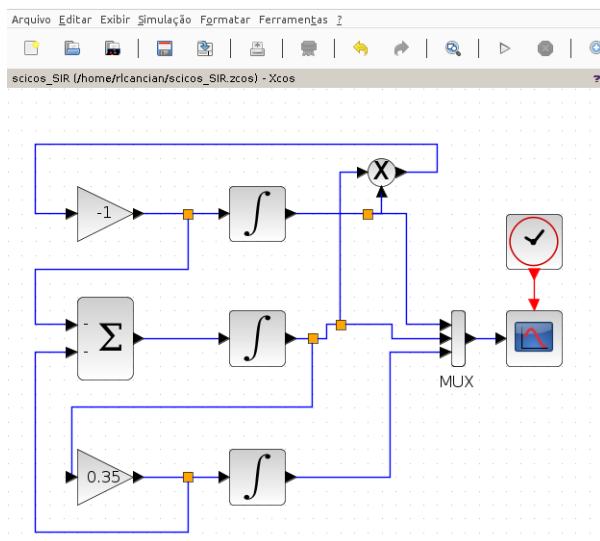


Figura 5.6: Modelo contínuo de disseminação de doença implementado no Xcos.

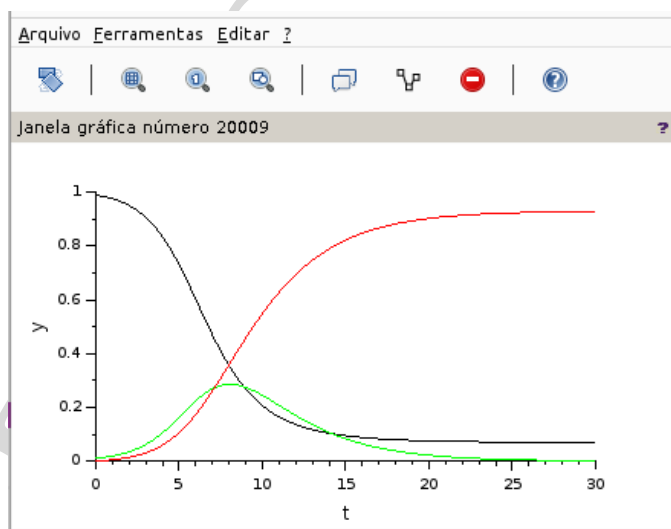


Figura 5.7: Resultado da simulação do modelo SIR no Xcos.

O comportamento global do modelo não é determinado apenas pelas conexões entre atores, mas também por um componente especial chamado *director*, que implementa o modelo de computação do nível correspondente.

A Figura 5.8 mostra a tela inicial do ambiente, enquanto a Figura 5.9 destaca a organização conceitual de diferentes modelos de computação associados ao ecossistema Ptolemy II.



Figura 5.8: Tela inicial do ambiente Ptolemy II.

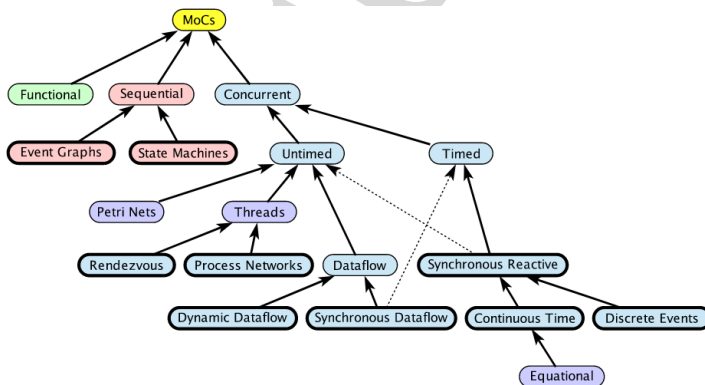


Figura 5.9: Exemplo de organização conceitual de modelos de computação no ecossistema Ptolemy II.

Uma representação particularmente importante da filosofia do ambiente aparece também na Figura 5.10, que ilustra a estrutura ator-orientada e a relação entre atores atômicos, atores compostos, portas, relações e diretores.

Como primeiro exemplo, o ambiente pode ser explorado com o sistema de Lorenz. A

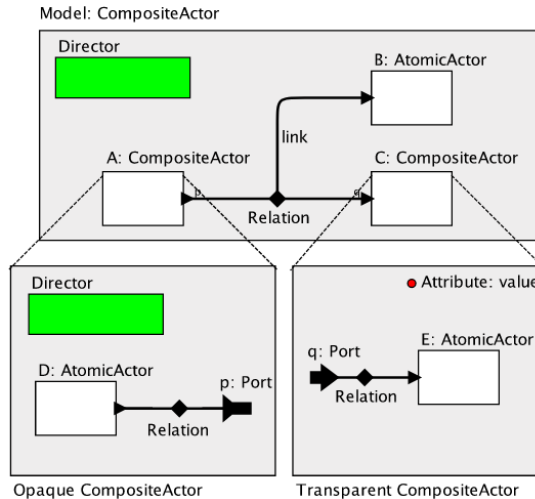


Figura 5.10: Modelo ator-orientado típico em Ptolemy II.

Figura 5.11 mostra um modelo montado no ambiente, enquanto a Figura 5.12 exibe a trajetória correspondente.

Do ponto de vista metodológico, o valor do ambiente vai muito além do fluxo operacional de abrir, selecionar atores, conectá-los e executar o modelo. Seu principal interesse está em tornar explícita a relação entre estrutura do modelo e semântica de execução, o que será particularmente importante nos capítulos 7, 8 e 9.

5.4.3 OpenModelica

OpenModelica é um ambiente aberto de modelagem e simulação associado à linguagem Modelica. Seu papel neste capítulo é importante porque permite distinguir, de maneira clara, a linguagem declarativa de modelagem do ambiente que a edita, traduz, compila, simula e analisa. Enquanto Modelica define a forma de expressar o modelo, OpenModelica oferece a infraestrutura necessária para trabalhar efetivamente com ele.

Do ponto de vista didático, o OpenModelica interessa especialmente porque aproxima o leitor do ecossistema declarativo multidomínio em uma plataforma aberta. Sua interface gráfica, OMEdit, permite carregar bibliotecas, inspecionar classes, montar modelos graficamente, editar código Modelica, executar simulações e plotar variáveis. A Figura 5.13 mostra uma captura representativa desse ambiente.

A forma mais simples de iniciar o uso do OpenModelica consiste em abrir o OMEdit, carregar a biblioteca padrão ou uma biblioteca adicional, inspecionar os componentes dispo-

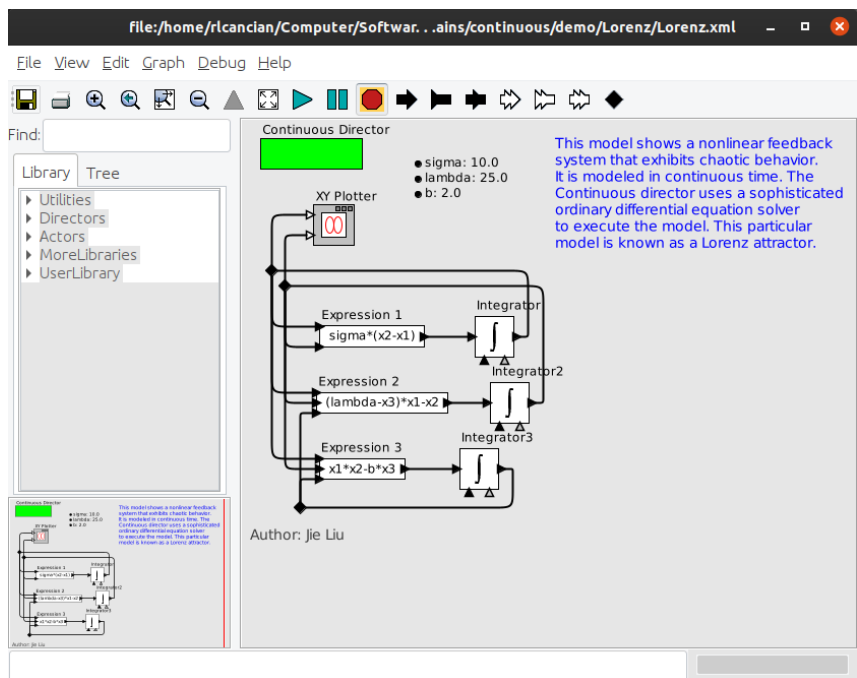


Figura 5.11: Modelo do atrator de Lorenz em Ptolemy II.

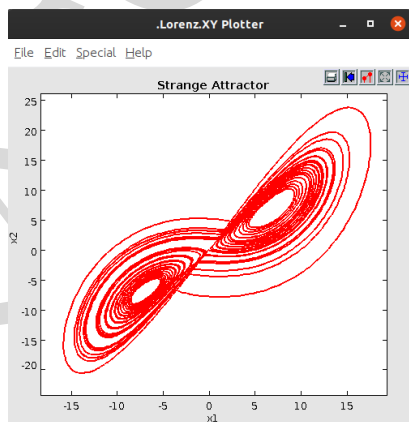


Figura 5.12: Trajetória do atrator de Lorenz gerada em Ptolemy II.

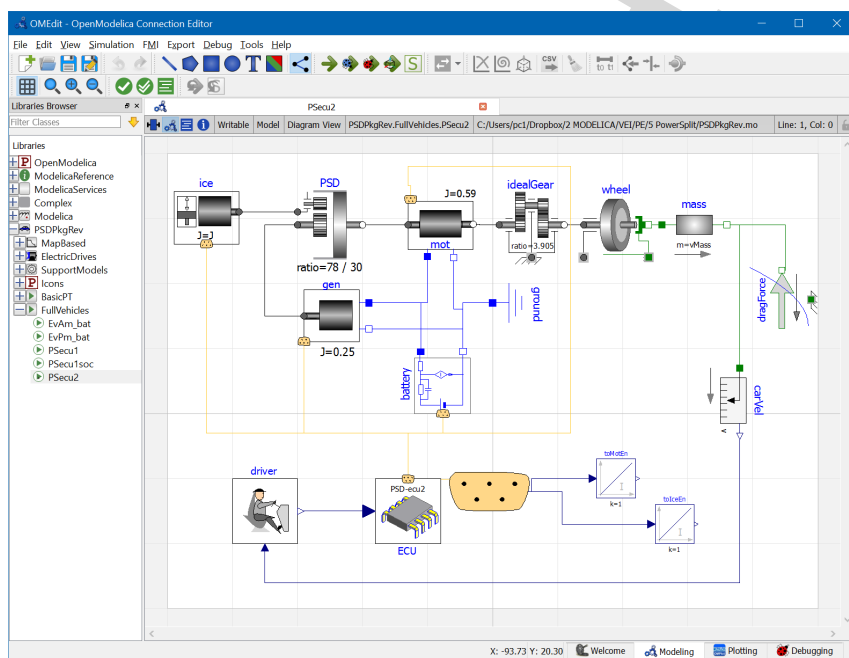


Figura 5.13: Exemplo de uso do OMEdit, ambiente gráfico do OpenModelica.

níveis e então construir ou abrir um modelo existente. Mais importante do que os detalhes de instalação, contudo, é compreender o papel conceitual da ferramenta: ela não substitui a linguagem Modelica, mas a torna operacional.

OpenModelica será um dos ambientes recorrentes dos capítulos técnicos posteriores, particularmente nos capítulos 7, 8 e 9, nos quais a linguagem declarativa e a modelagem de sistemas físicos e híbridos desempenharão papel central.

5.4.4 Arena

Arena é um dos ambientes de modelagem e simulação mais conhecidos na tradição da simulação discreta orientada a eventos. Sua importância metodológica decorre de dois fatos. O primeiro é que ele exemplifica, de forma bastante clara, a ideia de ambiente gráfico de alto nível voltado à modelagem por fluxo de entidades, recursos, filas e atividades. O segundo é que, apesar dessa interface gráfica, sua lógica subjacente está historicamente ancorada em SIMAN.

Arena não deve ser entendido apenas como um conjunto de desenhos conectados na tela. Ele organiza módulos, parâmetros, lógica de controle da execução, estatísticas automáticas, relatórios e infraestrutura experimental, mas tudo isso repousa sobre uma semântica mais profunda herdada da tradição de SIMAN. A observação dessa estrutura ajuda o leitor a compreender como um ambiente moderno de simulação pode encapsular uma linguagem clássica sem eliminá-la conceitualmente.

Uma captura típica do ambiente pode ser vista na Figura 5.14. A Figura 5.15 mostra especificamente o exemplo *Work in Process*, e a Figura 5.16 exhibe o relatório produzido pela execução correspondente.

Do ponto de vista didático, é particularmente útil abrir um dos modelos distribuídos com a própria ferramenta para explorar simultaneamente a interface gráfica, a animação, a lógica subjacente em módulos de processo e os relatórios automáticos gerados pela simulação. Essa exploração inicial prepara o terreno para o papel central que o Arena desempenhará no capítulo dedicado à modelagem discreta orientada a eventos.

5.4.5 GenESyS

GenESyS é *software* livre e de código aberto, concebido como simulador genérico e expansível. No contexto deste capítulo, sua importância é dupla. Em primeiro lugar, funciona como ambiente de simulação e experimentação. Em segundo lugar, permite ao leitor observar com maior transparência a estrutura interna de um simulador, pois foi projetado com forte preocupação didática e arquitetural.

Em sua concepção original, o GenESyS surgiu com foco na modelagem discreta baseada em fluxo de entidades e em componentes análogos aos do Arena, como *Create*, *Seize*, *Delay*, *Release* e *Dispose*. Posteriormente, sua arquitetura foi estendida para acomodar outras técnicas de modelagem e simulação. Essa evolução o torna relevante como ambiente

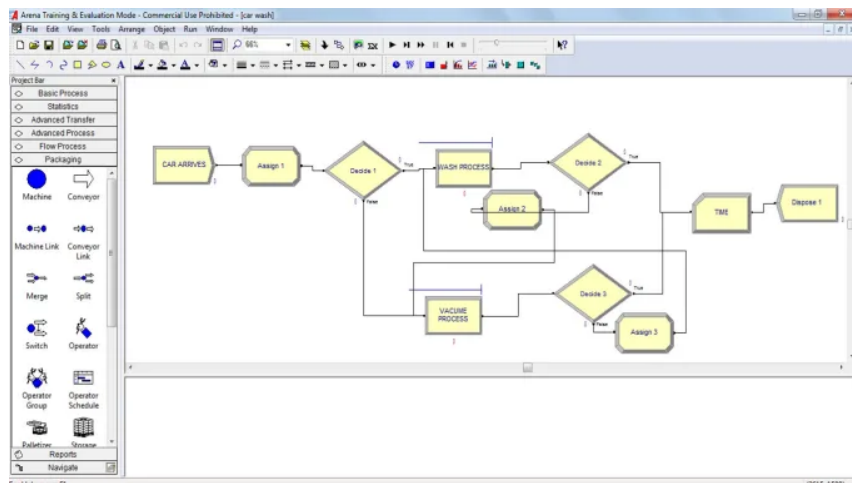


Figura 5.14: Exemplo de interface e modelo em Arena.

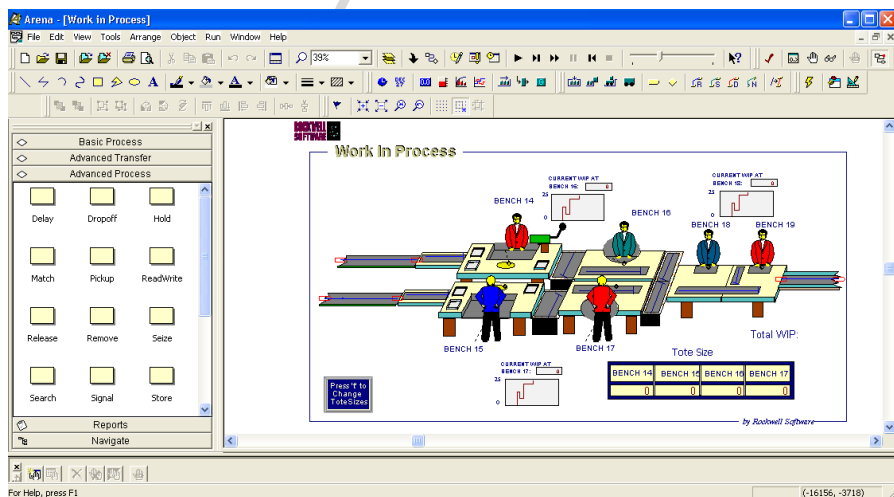


Figura 5.15: Modelo *Work in Process* aberto no Arena.

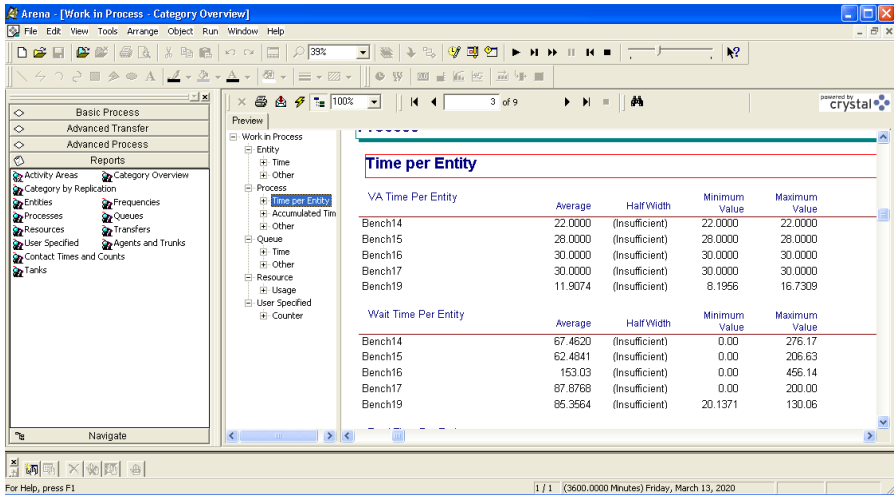


Figura 5.16: Relatório de saída associado ao modelo *Work in Process*.

e como estudo de caso, embora, novamente, não seja adotado como plataforma prática recorrente no restante deste volume.

A interface gráfica do ambiente, mostrada na Figura 5.17, evidencia sua aproximação com simuladores visuais de processo, sem que a estrutura interna da plataforma fique completamente escondida.

Para um primeiro uso didático, o GenESyS pode ser explorado abrindo um exemplo simples no ambiente visual, observando seus componentes e executando a simulação. Contudo, diferentemente do que ocorrerá com Ptolemy II, OpenModelica e Arena, essa exploração não terá continuidade sistemática nos próximos capítulos deste livro. Seu aprofundamento ficará reservado ao segundo volume.

Exemplo 5.4 Explique por que GenESyS permanece relevante neste capítulo, mesmo não sendo adotado como plataforma prática recorrente do restante do livro.

Solução: GenESyS permanece relevante porque ilustra de forma particularmente clara a coexistência entre linguagem de modelagem, biblioteca de simulação, simulador e ambiente gráfico. Além disso, ele faz parte do panorama de tecnologias da área e funciona como estudo de caso arquitetural importante, ainda que seu uso prático recorrente fique reservado a outro volume.

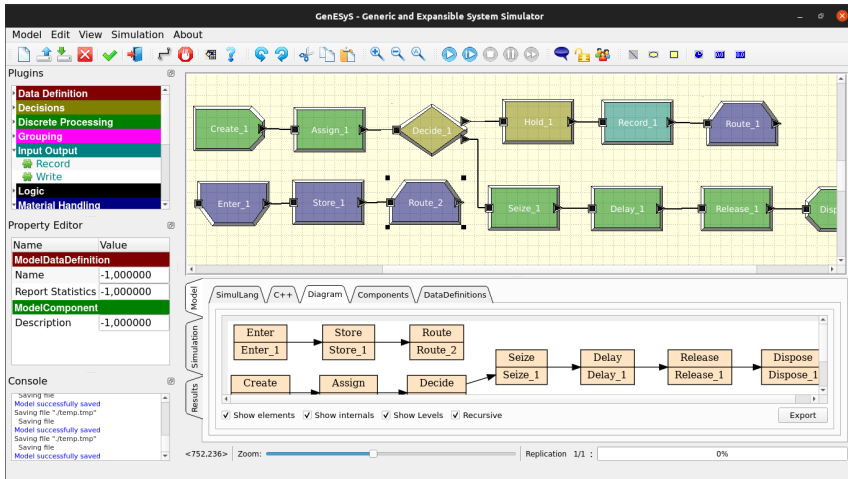


Figura 5.17: Exemplo da interface gráfica do GenESyS.

5.5 Critérios para escolha de linguagem, biblioteca ou ambiente

A escolha da tecnologia computacional adequada para um projeto de modelagem e simulação deve ser entendida como decisão multicritério. Não existe uma ferramenta universalmente melhor. Existem, isso sim, ferramentas mais ou menos adequadas a determinado tipo de sistema, paradigma, equipe, infraestrutura e objetivo científico ou de engenharia.

Um primeiro grupo de critérios diz respeito à relação entre problema e formalismo. A ferramenta escolhida deve ser compatível com a natureza do tempo, do estado e da dinâmica do sistema. Se o problema exige modelagem contínua multidomínio baseada em equações e componentes físicos, linguagens declarativas e ambientes associados a esse estilo de modelagem tendem a ser mais naturais. Se o foco está em entidades, filas, recursos e eventos, ambientes orientados à simulação discreta por eventos costumam ser mais adequados. Se a questão central envolve composição semântica entre paradigmas, ambientes heterogêneos tornam-se especialmente relevantes.

Um segundo grupo de critérios está associado à equipe e ao processo de desenvolvimento. Ferramentas muito poderosas do ponto de vista conceitual podem ser inadequadas se a equipe não possuir formação suficiente para utilizá-las com segurança. Da mesma forma, tecnologias abertas e programáveis podem ser excelentes em projetos de pesquisa, mas inadequadas quando o prazo exige forte produtividade imediata e pouca liberdade de customização. Curva de aprendizado, maturidade da equipe, disponibilidade de exemplos e clareza da documentação precisam ser levadas em conta.

Um terceiro grupo de critérios diz respeito à integração. Modelos raramente vivem isolados. Em muitos projetos, o simulador precisa interagir com bases de dados, ferramentas estatísticas, bibliotecas numéricas, ambientes de visualização, sistemas legados, serviços em rede ou plataformas de otimização. Nesses casos, a abertura do ecossistema, a existência de APIs, o suporte a troca de arquivos e a interoperabilidade tornam-se decisivos.

Há ainda critérios clássicos de engenharia de *software* e gestão de projetos: custo de licenciamento, disponibilidade para o sistema operacional de interesse, suporte técnico, sustentabilidade tecnológica do ecossistema, transparência interna da ferramenta, possibilidades de depuração e reprodutibilidade dos experimentos. Em contexto acadêmico, critérios como abertura, capacidade de inspeção da lógica subjacente e adequação pedagógica também assumem peso especial.

A figura 5.18 organiza esses critérios em um quadro sintético. Ela não fornece uma regra automática de decisão, mas ajuda a estruturar o julgamento técnico do modelador.

Grupo de critério	Pergunta orientadora	Exemplos de decisão
Problema e formalismo	A tecnologia representa naturalmente o paradigma escolhido?	contínuo, discreto, híbrido, multidomínio
Equipe e aprendizado	A equipe domina a ferramenta e seu custo de adoção?	curva de aprendizado, didatismo, exemplos
Integração	A ferramenta conversa com o restante do ecossistema?	APIs, arquivos, bibliotecas, dados
Execução e análise	A execução, depuração e análise são adequadas ao projeto?	relatórios, animação, visualização, repetições
Projeto e sustentabilidade	A solução é viável no contexto institucional?	licença, plataforma, suporte, manutenção

a escolha da tecnologia é multicritério e subordinada ao problema

Figura 5.18: Critérios metodológicos, técnicos e práticos para escolha de tecnologia computacional de modelagem e simulação.

Uma forma útil de resumir a posição deste capítulo é dizer que o problema deve dominar a escolha da tecnologia, e não o contrário. Essa conclusão aprofunda a discussão do capítulo anterior: o paradigma de modelagem orienta a escolha da tecnologia, mas a tecnologia também pode retroagir sobre a forma prática de desenvolver o modelo. O equilíbrio entre esses dois movimentos é parte da maturidade metodológica do projeto.

Exemplo 5.5 Explique por que uma ferramenta muito poderosa do ponto de vista computacional pode ainda assim ser inadequada para um determinado projeto de simulação.

Solução: Uma ferramenta pode ser muito poderosa e, ainda assim, inadequada se o paradigma que implementa não for o mais apropriado ao problema, se a equipe não tiver domínio suficiente para utilizá-la, se o custo de adoção for incompatível com o projeto ou se ela não se integrar bem ao restante do ecossistema computacional necessário ao estudo.

5.6 Considerações Finais

Este capítulo discutiu os principais meios computacionais pelos quais modelos podem ser descritos, implementados, executados e analisados. Com isso, tornou-se possível distinguir com mais precisão categorias frequentemente confundidas na prática, como linguagem de programação, biblioteca, *framework*, linguagem clássica de simulação, linguagem declarativa de modelagem e ambiente integrado. Essa distinção não é apenas terminológica. Ela afeta diretamente a forma como o modelo é expresso, a infraestrutura que a tecnologia oferece, o esforço de desenvolvimento exigido da equipe e o tipo de raciocínio metodológico envolvido na construção do estudo.

Também se evidenciou que a evolução histórica da área não eliminou os fundamentos conceituais das tecnologias anteriores. Ao contrário, muitos ambientes contemporâneos encapsulam, estendem ou reorganizam ideias já presentes em linguagens clássicas como GPSS e SIMAN, ao mesmo tempo em que incorporam mecanismos modernos de composição, reutilização, experimentação e integração com outros ecossistemas computacionais. Assim, compreender linguagens, bibliotecas e simuladores como artefatos arquiteturalmente distintos, mas metodologicamente relacionados, é parte essencial da formação do modelador.

Por fim, o capítulo reforçou que a escolha da tecnologia computacional deve ser subordinada ao paradigma de modelagem, ao domínio do problema, ao objetivo do estudo e às restrições práticas do projeto. Em outras palavras, o problema deve dominar a escolha da linguagem, da biblioteca ou do ambiente, e não o contrário. O próximo capítulo dará continuidade à metodologia do projeto de simulação, tratando da coleta, preparação e análise de dados de entrada. Além disso, informações complementares sobre o simulador GenESyS serão apresentadas no Anexo II.

5.7 Resumo

Seção 5.1 Introdução

- Modelos conceituais e paradigmas de modelagem precisam de artefatos computacionais específicos para serem implementados, executados, depurados e analisados.
- Linguagem, biblioteca, *framework*, simulador, plataforma e ambiente são categorias relacionadas, mas não equivalentes.

- Este livro utilizará como plataformas práticas recorrentes apenas Ptolemy II, OpenModelica e Arena.
- Scilab/Xcos e GenESyS permanecem relevantes neste capítulo como parte do panorama de tecnologias da área.

Seção 5.2 Taxonomia de tecnologias computacionais para modelagem e simulação

- Linguagens de programação de propósito geral oferecem alta flexibilidade, mas exigem que a infraestrutura de simulação seja construída ou integrada explicitamente.
- Bibliotecas e *frameworks* de simulação estendem uma linguagem hospedeira com mecanismos para tempo, eventos, recursos, filas, processos e estatísticas.
- Linguagens clássicas de simulação possuem sintaxe e semântica próprias, diretamente orientadas à descrição de modelos e, em muitos casos, também de experimentos.
- Linguagens declarativas de modelagem descrevem relações, equações, componentes e conexões, em vez de programar explicitamente a ordem imperativa de execução.
- Ambientes de modelagem e simulação integram editor, bibliotecas de componentes, motor de simulação, depuração, parametrização experimental, visualização e análise.

Seção 5.3 Linguagens de simulação e de modelagem representativas

- GPSS tornou explícita a lógica de fluxo de entidades por blocos e consolidou uma forma histórica de representar sistemas discretos orientados a transações, filas e recursos.
- SIMAN separa estruturalmente a lógica do modelo e o plano experimental, reforçando que modelo e experimento são artefatos relacionados, mas distintos.
- Modelica é linguagem declarativa e acausal, apropriada para sistemas físicos e multidomínio descritos por componentes, equações e conexões.
- GenESyS pode ser visto simultaneamente como linguagem de modelagem, API programável, simulador e plataforma expansível.

Seção 5.4 Ambientes e simuladores relevantes para a obra

- Scilab e Xcos combinam processamento numérico, diagramação por blocos e simulação contínua, permitindo integrar modelagem gráfica e análise computacional em um mesmo ecossistema.
- Ptolemy II organiza explicitamente diferentes modelos de computação e destaca a relação entre estrutura do modelo, atores, diretores e semântica de execução.
- OpenModelica integra linguagem Modelica, bibliotecas, edição textual e gráfica, simulação e visualização de resultados em um ambiente declarativo multidomínio aberto.
- Arena combina editor gráfico, módulos de processo, motor de simulação, relatórios e ferramentas auxiliares, mantendo ligação conceitual profunda com a linguagem SIMAN.
- GenESyS permanece relevante como ambiente e estudo de caso arquitetural, embora seu uso prático recorrente seja reservado a outro volume.

Seção 5.5 Critérios para escolha de linguagem, biblioteca ou ambiente

- A escolha da tecnologia computacional é decisão multicritério e não admite solução

universalmente ótima para todos os projetos.

- Compatibilidade entre problema e formalismo é critério central, porque a ferramenta deve respeitar a natureza do tempo, do estado e da dinâmica do sistema estudado.
- Curva de aprendizado, maturidade da equipe, clareza da documentação e produtividade prática influenciam diretamente a adequação da tecnologia ao projeto.
- Integração com bases de dados, bibliotecas numéricas, ferramentas estatísticas, visualização, sistemas legados e serviços externos pode ser decisiva na escolha.
- Custo, licenciamento, abertura do ecossistema, depuração, reprodutibilidade e valor pedagógico também são critérios relevantes na seleção de linguagens, bibliotecas e ambientes.

5.8 Exercícios Propostos

Conceituação

Exercício 5.1 Quais são as diferenças entre uma linguagem de programação de propósito geral, uma biblioteca de simulação, uma linguagem clássica de simulação, uma linguagem declarativa de modelagem e um ambiente de modelagem e simulação?

Exercício 5.2 Explique por que Modelica deve ser classificada como linguagem declarativa de modelagem, e não como simulador.

Exercício 5.3 Explique por que OpenModelica deve ser classificado como ambiente de modelagem e simulação, e não apenas como linguagem.

Exercício 5.4 Discuta a diferença entre uma linguagem clássica de simulação, como GPSS ou SIMAN, e uma abordagem moderna baseada em *framework*.

Exercício 5.5 Explique por que GenESyS é um bom exemplo de que linguagem de simulação, biblioteca de simulação e simulador não são categorias mutuamente exclusivas.

Comparação entre tecnologias

Exercício 5.6 Compare GPSS e SIMAN em termos de estilo de modelagem, expressividade, domínio de aplicação e relação com ambientes modernos.

Exercício 5.7 Compare Xcos, Ptolemy II e OpenModelica quanto ao tipo de sistema para o qual cada ambiente é mais adequado, ao grau de heterogeneidade suportado e à relação entre representação gráfica e formalismo subjacente.

Exercício 5.8 Compare Arena e GenESyS quanto ao paradigma principal suportado, ao grau de abertura do ecossistema, à possibilidade de inspeção da lógica subjacente e à vocação didática.

Aplicação

Exercício 5.9 Abra um exemplo simples em Ptolemy II, modifique ao menos um parâmetro do modelo e descreva qualitativamente o efeito observado sobre os resultados.

Exercício 5.10 No Xcos, modifique o modelo SIR apresentado neste capítulo para incluir uma nova variável de estado x_4 , representando a fração de indivíduos mortos, com taxa de mortalidade $\gamma x_2(t)$. Reescreva o sistema de equações correspondente e simule o novo modelo.

Exercício 5.11 No Arena, abra um exemplo da biblioteca padrão, identifique a lógica principal do modelo e descreva quais módulos correspondem a geração de entidades, filas, recursos, atrasos e descarte.

Exercício 5.12 No OpenModelica, construa ou abra um modelo simples da biblioteca padrão, execute a simulação e identifique a classe Modelica correspondente, os principais parâmetros do modelo e as variáveis disponíveis para plotagem.

Exercício 5.13 No GenESyS, execute um exemplo simples e identifique os principais componentes do modelo, o tipo de entidade utilizado, as expressões probabilísticas definidas e as estatísticas produzidas pelo simulador.

Investigação

Exercício 5.14 Escolha uma biblioteca moderna de simulação baseada em linguagem de propósito geral e discuta como ela se posiciona em relação às categorias apresentadas neste capítulo.

Exercício 5.15 Investigue um ambiente contemporâneo multimétodo e compare-o com as arquiteturas de Xcos, Ptolemy II e OpenModelica.

Exercício 5.16 Pesquise a documentação de uma linguagem clássica de simulação e discuta quais conceitos ainda permanecem visíveis em simuladores gráficos modernos.

Exercício 5.17 Escolha um sistema de sua área de interesse e proponha, justificadamente, uma combinação de paradigma de modelagem e tecnologia computacional adequada para estudá-lo.

Exercício 5.18 Discuta criticamente até que ponto uma ferramenta aberta e expansível pode ter vantagem metodológica sobre uma ferramenta proprietária fechada em contexto de ensino e pesquisa.

Base bibliográfica do capítulo. Este capítulo se apoia principalmente em [29], [24], [42], [40], [33], [36], [14], [47] e [16] para discutir a taxonomia de linguagens, bibliotecas, *frameworks*, ambientes e simuladores de modelagem e simulação. A discussão histórica e metodológica sobre linguagens clássicas de simulação também se apoia em [45] e [37]. Já a apresentação do GenESyS como linguagem, simulador e ambiente de modelagem dialoga com [9], mantendo sua presença neste capítulo como parte do panorama de tecnologias da área, embora seu uso prático sistemático fique reservado a outro volume.